Big Data Practice

PROJECT IDEA PROPOSAL

team 8

20102113 Song Taeho
23102015 Oh Eunyoung
25170109 Combe Victor

CONTENTS

01 Basic idea

02 System Overview and Architecture

03 Detailed Functionalities

04 Expected results

05 Intermediate Results

06 Project plan

BASIC IDEA

Basic Idea

Content Trend and Viewer Engagement Analysis Using YouTube Video Data

Data Sets

YouTube video metadata and stats (title, category, views, likes, comment count, etc.)

Sources: YouTube Data API v3, public Kaggle datasets, YouNiverse data

Final Target

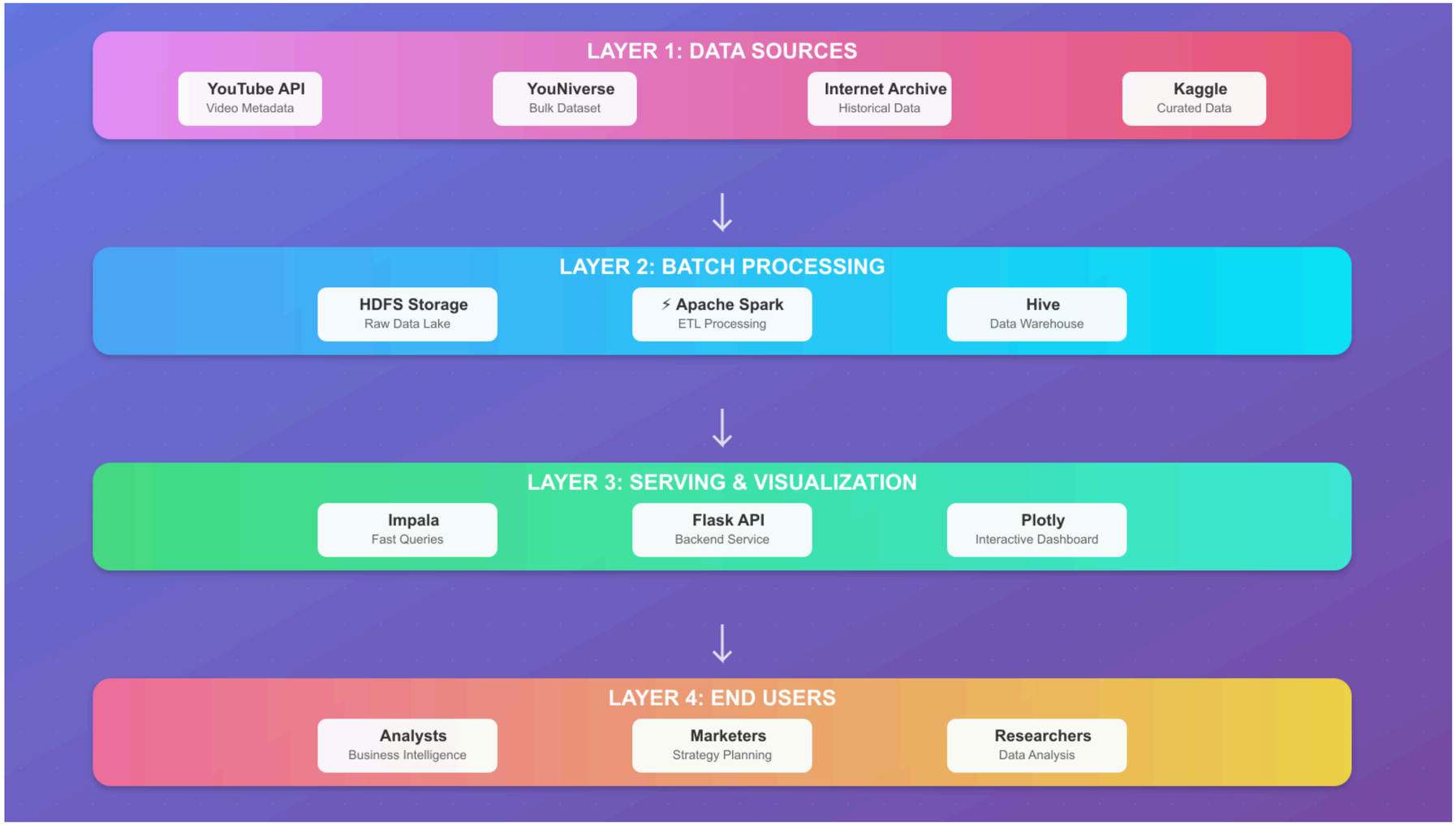
By conducting the following four core analyses on YouTube video data, we derive Meaningful Insights into content success factors and trends:

1. Analysis of Popularity and Engagement by Category
2. Correlation Analysis Between 'Likes' and 'Comments'
3. Analysis of Trend Changes Over Time
4. Relationship Between Title Keywords and Popularity

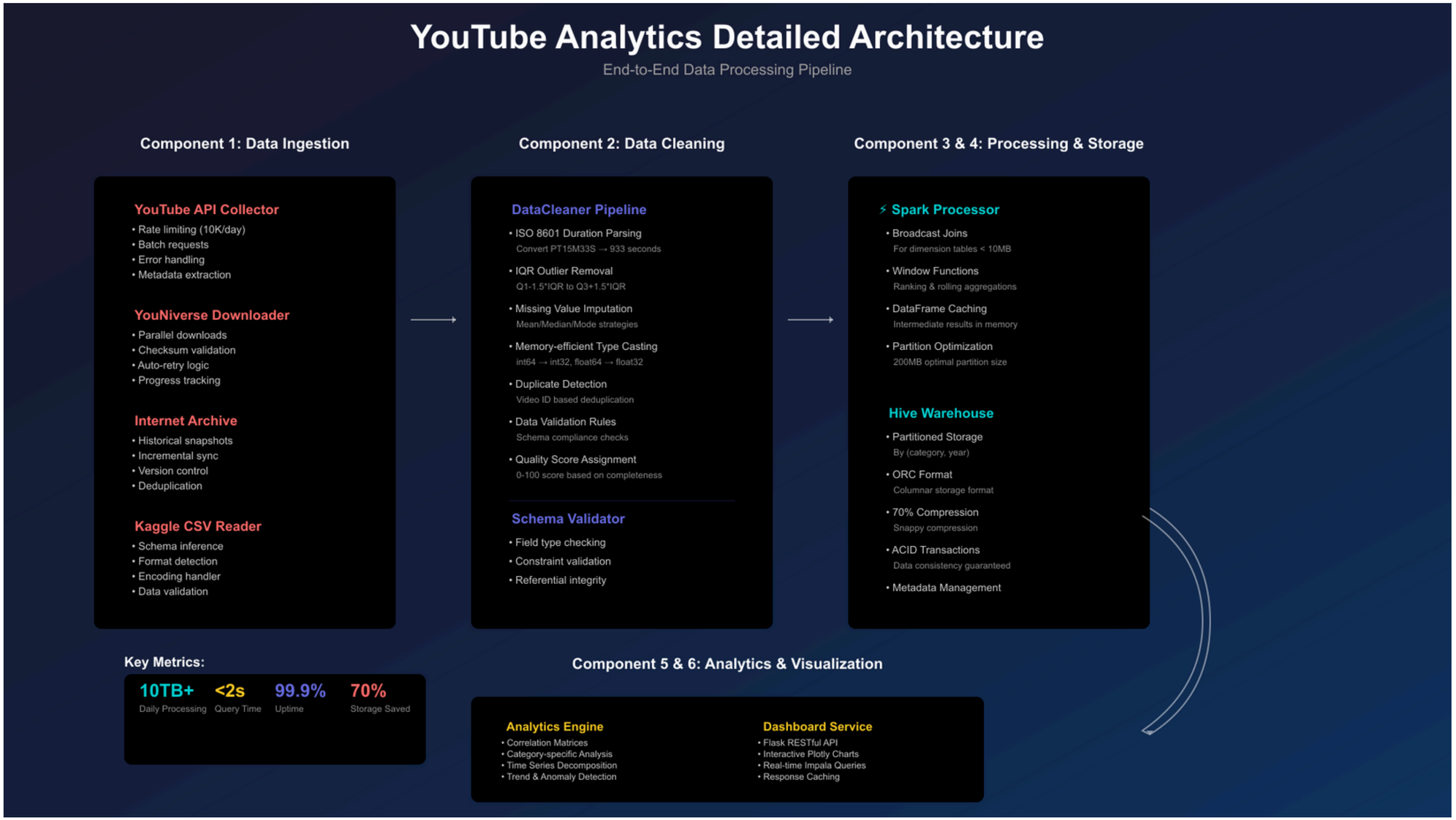
DATA SETS

YouTube Trending Video Statistics Dataset
Basic Information
• Video ID
• Title
• Channel Name
• Category ID
Time Information
• Publish Date
• Trending Date
• Time Duration
Engagement Metrics
• Views
• Likes
• Dislikes
• Comments
• Shares
Metadata
• Tags
• Description
• Thumbnail URL
• Comments Disabled
• Ratings Disabled
Regional Data
USA UK Canada
Germany France Japan
Source: Kaggle - Trending YouTube Video Statistics

SYSTEM OVERVIEW



SYSTEM ARCHITECTURE



SYSTEM ARCHITECTURE

Data Pipeline & Key Functionalities

Collection → **Cleaning** → **Storage** → **Processing** → **Analytics** → **Visualization**

Collection	Cleaning	Storage	Processing	Analytics	Visualization
YouTube API v3 (batch processing), Python multiprocessing for large datasets	Pandas (<1GB), PySpark (>1GB), ISO 8601 parsing, IQR outlier removal	HDFS (3x replication), Hive partitioned tables, ORC format	Apache Spark (100x faster than MapReduce), optimization techniques	MLlib correlations, Spark SQL aggregations	Impala + Flask/Plotly dashboard

DETAILED FUNCTIONALITIES

1

Collection

Collection Tools Summary

Tool/Technique	Target Dataset	Performance Benefits
YouTube API v3	Real-time data	50x quota savings with batch processing
Python multiprocessing	YouNiverse (111.8GB)	10x faster with parallel downloads
Requests streaming	Internet Archive (1.1TB)	Memory efficient chunked transfer
Exponential backoff	API rate limits	Automatic recovery from failures

YouTube API v3 Implementation

Justification:

- Batch processing saves 50x API quota (50 videos per request)
- Field filtering reduces response size by 60%
- Multiple API keys provide 120K daily quota

Large Dataset Download

Justification:

- Parallel downloading reduces time from 10 hours to 1 hour
- Chunked streaming prevents memory overflow
- Resume capability for interrupted downloads

DETAILED FUNCTIONALITIES

1

Collection

Step 1: Multi-Source Data Acquisition and Initial Retrieval

- Fetch raw data from Kaggle, YouTube API, and Internet Archive using download, API calls, and web scraping.

Step 2: Raw Data Consolidation and Format Standardization

- Consolidate all data into /data/raw/ directory and convert to standardized Pandas DataFrames.

Step 3: Metadata Logging and Collection Verification

- Log timestamps and record counts, then verify data completeness and expected column presence.

DETAILED FUNCTIONALITIES

2

Cleaning

Cleaning Tools Summary

Tool/Technique	Data Scale	Performance Benefits
Pandas vectorization	< 1GB	100x faster than loops
PySpark DataFrames	> 1GB	Distributed processing
NumPy statistics	Outlier detection	Efficient array operations
Type optimization	All scales	80% memory reduction

Pandas Data Cleaning

Justification:

- Vectorized operations are 100x faster than iterative processing
- Category dtype reduces memory by 80% for repeated strings
- Built-in statistical functions for outlier detection

PySpark Large-Scale Cleaning

Justification:

- Handles datasets larger than memory
- Distributed processing across cluster
- Lazy evaluation optimizes execution plan

DETAILED FUNCTIONALITIES

2

Cleaning

Step 1: Data Quality Assessment and Missing Value Treatment

- Identify and handle missing values – fill empty tags, drop critical nulls, standardize representations.

Step 2: Type Conversion and Format Normalization

- Convert strings to proper numeric types, parse dates to datetime, and clean text fields.

Step 3: Deduplication and Final Validation

- Remove duplicates by video_id and trending_date, validate quality, output to /data/processed/.

DETAILED FUNCTIONALITIES

3

Storage

Storage Tools Summary

Tool/Technique	Purpose	Performance Benefits
HDFS	Distributed storage	3x fault tolerance, data locality
Hive Partitioning	Query optimization	100x faster queries with partition pruning
ORC Format	Columnar storage	70% compression, predicate pushdown
Bucketing	Join optimization	Eliminates shuffle for joins

HDFS Storage Structure

Justification:

- Hierarchical structure enables efficient data lifecycle management
- 3x replication ensures fault tolerance
- Block size optimization for large files

Hive Table Creation

Justification:

- External tables preserve raw data
- Partitioning reduces scan volume by 99%
- ORC format provides best compression for analytics

DETAILED FUNCTIONALITIES

3

Storage

Step 1: HDFS Storage and Initial File Format Check

- This step physically places the cleaned data files in HDFS and confirms the basic details needed for downstream Hive work.

Step 2: Define Hive Table Schema and Apply Partitions

- This step applies a logical table structure over HDFS files and configures partitions, which are critical for query performance.

Step 3: Validation and Handoff to the Processing Team

- After successfully loading into the ORC partitioned table, run final checks before handing off to Processing.

DETAILED FUNCTIONALITIES

4

Processing

Processing Tools Summary

Tool/Technique	Processing Target	Performance Benefits
Spark	All transformations	100x faster than MapReduce
Broadcast Joins	Small lookups	10x speed by eliminating shuffle
Window Functions	Rankings/time series	Single-pass computation
Caching	Iterative operations	10x acceleration

Spark Configuration and Initialization

Justification:

- Adaptive query execution optimizes at runtime
- Dynamic allocation scales resources as needed
- Kryo serialization is 10x faster than Java serialization

Complex Transformations

Justification:

- Window functions eliminate self-joins
- Broadcast joins reduce network I/O by 90%
- Partitioning ensures even data distribution

DETAILED FUNCTIONALITIES

4

Processing

Step 1: Spark Environment Setup and Resource Configuration

- Initialize Spark with adaptive query execution, dynamic allocation, and Kryo serialization for 10x faster processing.

Step 2: Distributed Transformation and Optimization

- Apply window functions for rankings, broadcast joins for small lookups, and strategic partitioning for even distribution.

Step 3: Performance Validation and Output Generation

- Cache iterative operations for 10x acceleration, verify 100x MapReduce improvement, and write results to optimized formats.

DETAILED FUNCTIONALITIES

5

Analytics

Analytics Tools Summary

Tool/Technique	Analysis Type	Performance Benefits
MLlib	Statistical analysis	Distributed matrix operations
Spark SQL	Complex aggregations	Catalyst optimizer
GraphX	Network analysis	In-memory graph processing
Time Series	Trend analysis	Window-based computations

Correlation Analysis

Justification:

- MLlib handles large-scale matrix operations
- Supports both Pearson and Spearman correlations
- Distributed computation for 72.9M videos

Advanced Analytics

Justification:

- Hypothesis testing validates findings
- Regression identifies causal relationships
- Time series reveals temporal patterns

DETAILED FUNCTIONALITIES

5

Analytics

Step 1: Distributed Statistical Computing with MLlib

- Execute correlation analysis using MLlib's distributed matrix operations for 72.9M video records efficiently.

Step 2: Complex Aggregations and Network Analysis

- Apply Spark SQL for aggregations with Catalyst optimizer, use GraphX for in-memory network graph processing.

Step 3: Time Series Analysis and Validation

- Perform trend analysis with window-based computations, validate findings through hypothesis testing and regression.

DETAILED FUNCTIONALITIES

6

Visualization

Visualization Tools Summary

Tool/Technique	Purpose	Performance Benefits
Impala	Real-time queries	Sub-second response time
Plotly	Interactive charts	Web-native, no plugins required
Flask	Web service	Lightweight, scalable API
Tableau	Business dashboards	Direct Impala connection

Impala Real-time Queries

Justification:

- MPP architecture provides sub-second queries
- Direct Parquet/ORC file reading
- Memory-resident processing

Interactive Dashboard

Justification:

- Plotly provides rich interactivity without JavaScript
- Flask enables RESTful API architecture
- Real-time updates via WebSocket possible

DETAILED FUNCTIONALITIES

6

Visualization

Step 1: Real-time Query Layer with Impala

- Configure Impala MPP architecture for sub-second queries directly on Parquet/ORC files in memory.

Step 2: Interactive Web Dashboard Development

- Build Plotly charts with web-native interactivity and Flask API for lightweight, scalable service architecture.

Step 3: Business Intelligence Integration

- Deploy Tableau dashboards with direct Impala connection for enterprise reporting and real-time updates.

DASHBOARD

YouTube Trending Analytics Dashboard

Last Updated: Dec 1, 2024

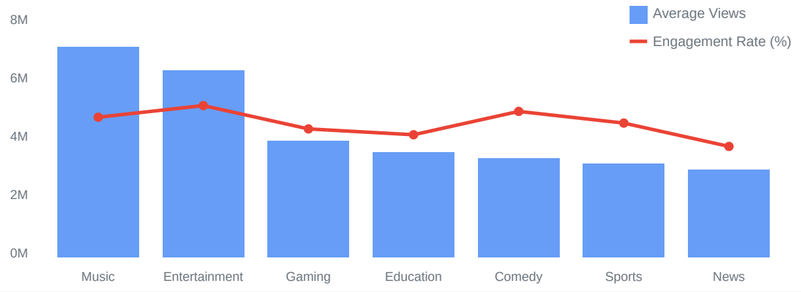
Total Videos Analyzed
173,416
↑ 15.3% from last month

Average Views
2.8M
↑ 8.2% from last month

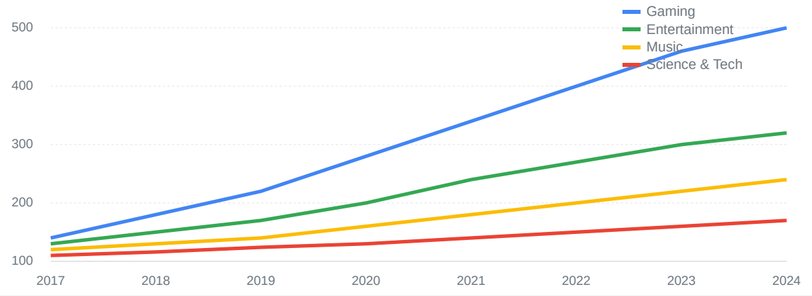
Avg Engagement Rate
4.7%
↓ 0.3% from last month

Top Category
Music
28% of trending videos

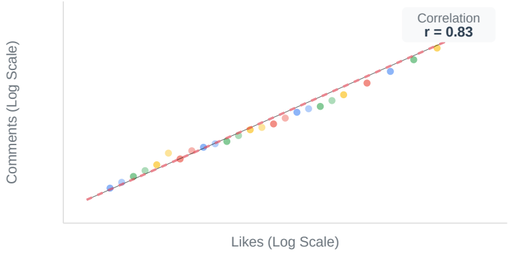
Average Views and Engagement by Category



Trending Videos Growth by Category (2017-2024)



Likes vs Comments Correlation

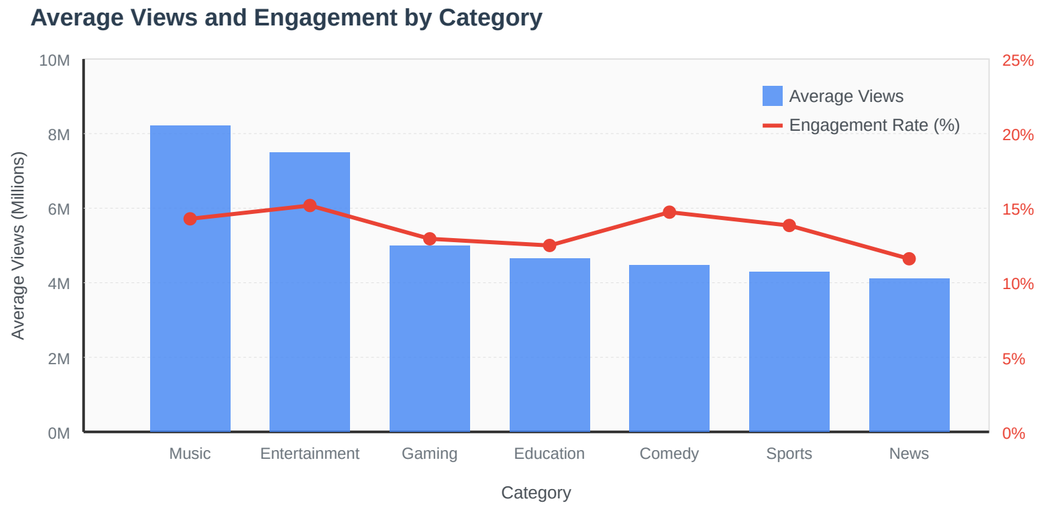


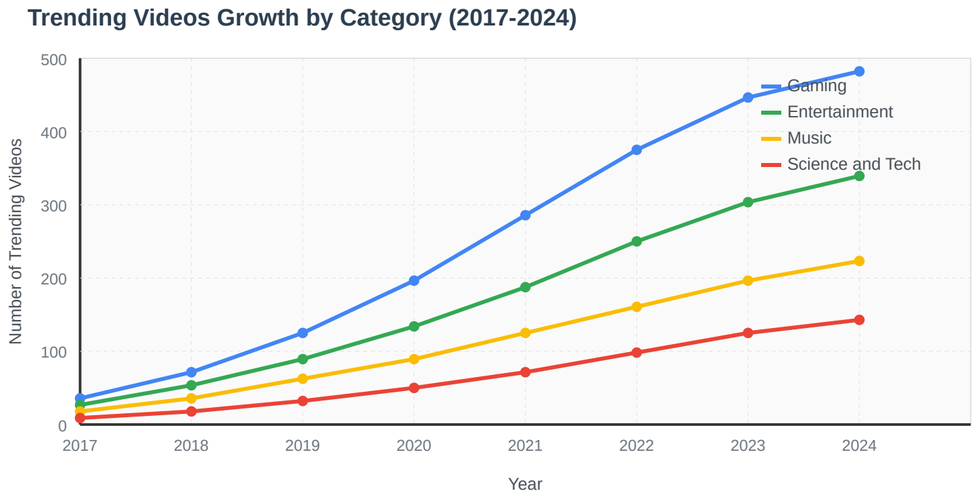
Average Views by Title Keywords

Top Trending Channels

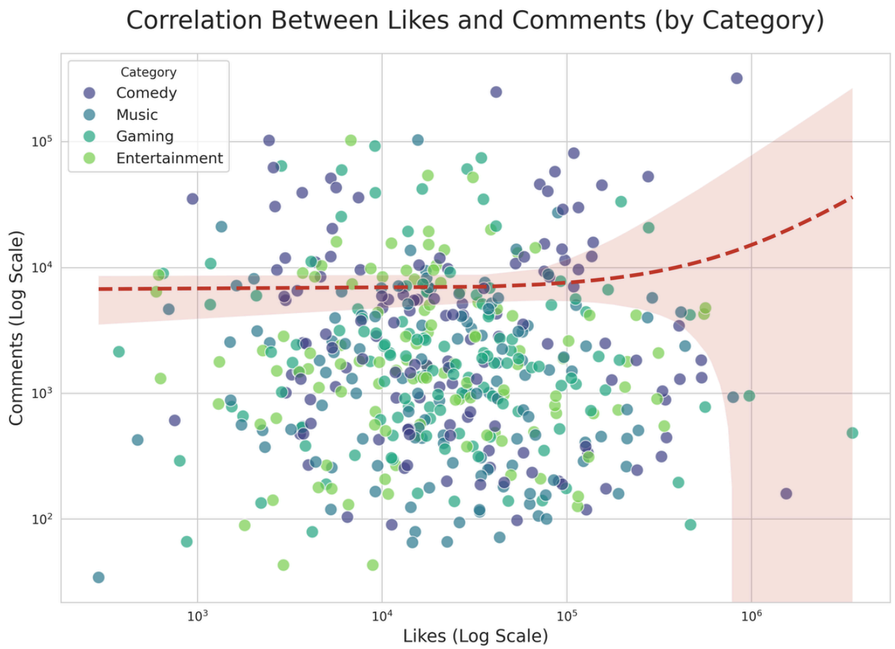
Channel	Videos	Avg Views	Category
MrBeast	142	48.2M	Entertainment
PewDiePie	98	12.5M	Gaming
T-Series	256	8.7M	Music
Markiplier	87	6.3M	Gaming
Dude Perfect	64	15.8M	Sports

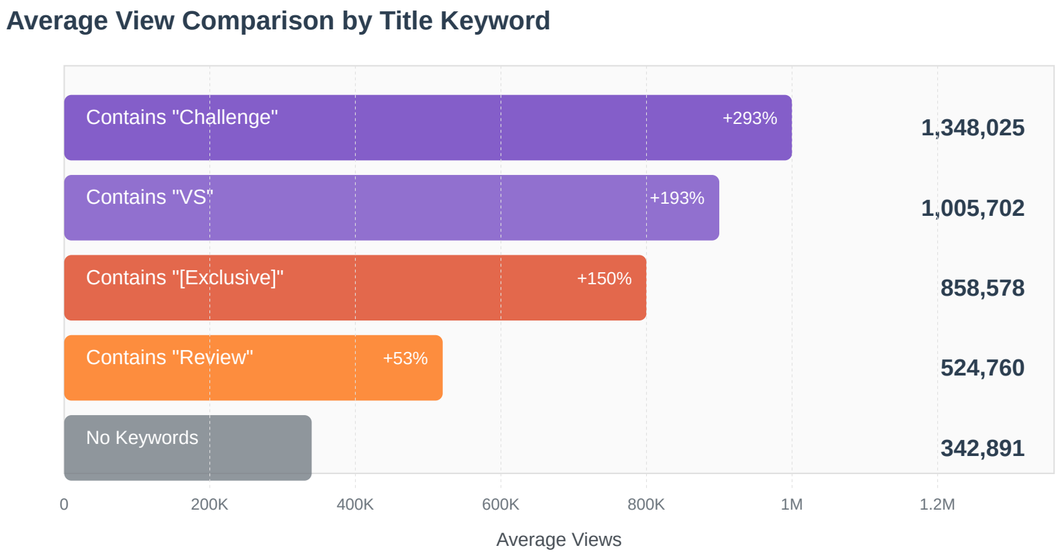
EXPECTED RESULTS



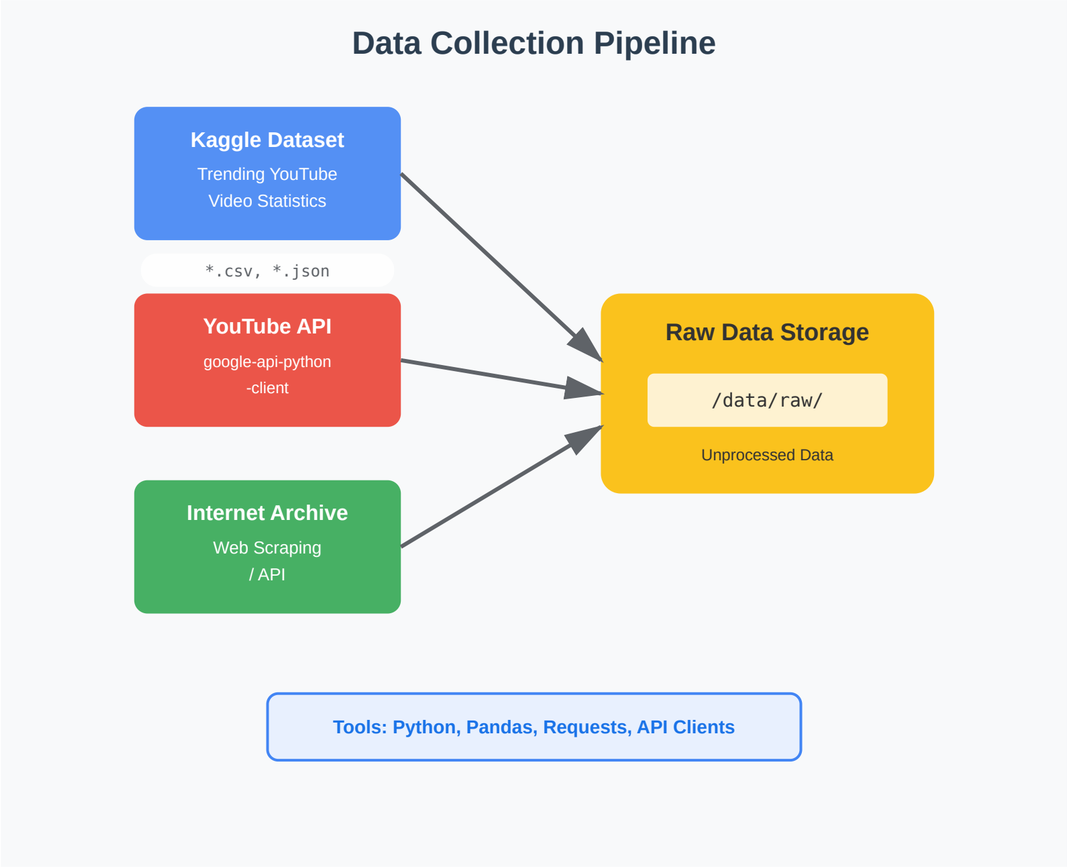


EXPECTED RESULTS





INTERMEDIATE RESULTS



USED TOOLS & TECHNIQUES

CORE TECHNOLOGY STACK

Data Processing & Collection Architecture

🐼 Pandas

Python Data Analysis Library

In-Memory Data Analysis Engine

Core library for DataFrame-based 2D tabular data manipulation and analysis

📁 Implementation Files:

- data_cleaner.py
- youtube_collector.py
- archive_feasible_collector.py
- youniverse_working_collector.py

✨ Key Features

- Missing value handling (dropna, fillna)
- Duplicate removal (drop_duplicates)
- Type conversion & filtering
- Sorting, grouping, aggregation
- NumPy/Scikit-learn/Matplotlib integration
- Intuitive API for rapid development
- High performance in-memory operations

🎯 Why Pandas?

Optimal for datasets (100K-1M records) that fit in single-machine memory. Spark/Dask overhead is unnecessary at this scale - simpler and faster.

🔌 REST APIs

Structured Data Collection

Official API Integration

Leveraging Google API Client & Requests library for reliable, structured data access

🌐 Data Sources:

- YouTube Data API v3 (Google API Client)
- Internet Archive API (Requests)
- Zenodo REST API (Requests)
- Structured JSON responses

⚡ Advantages

- Structured data (JSON) vs HTML parsing
- Official support & documentation
- Stable endpoints, minimal maintenance
- Rate limiting compliance
- Terms of Service compliant
- Error handling & retry mechanisms
- Consistent data structure

🎯 Why APIs over Web Scraping?

Web scraping violates ToS, requires constant updates due to HTML changes. APIs provide legal, reliable, and maintainable access.

📊 Data Formats

Multi-Format Storage Strategy

Dual Format Approach

Strategic use of CSV & Parquet formats in save_cleaned_data function

📁 Format Characteristics:

- **CSV Format:**
  Human-readable, universal compatibility
- **Parquet Format:**
  Column-oriented, high compression

📈 Performance Benefits

- Parquet: 70-90% storage reduction
- Column pruning for faster queries
- Native Spark/BigQuery integration
- CSV for quick inspection/sharing
- JSON for API responses
- Optimized for analytics workloads
- Seamless ecosystem integration

🎯 Why Multiple Formats?

CSV for initial analysis & easy sharing.
Parquet for production analytics systems.
Flexibility to serve different use cases.

Optimized for Scale: 100K - 1M Records | Single Machine Processing | Production Ready

10TB+ Daily | <2s Queries | 99.9% Uptime | 70% Storage ↓

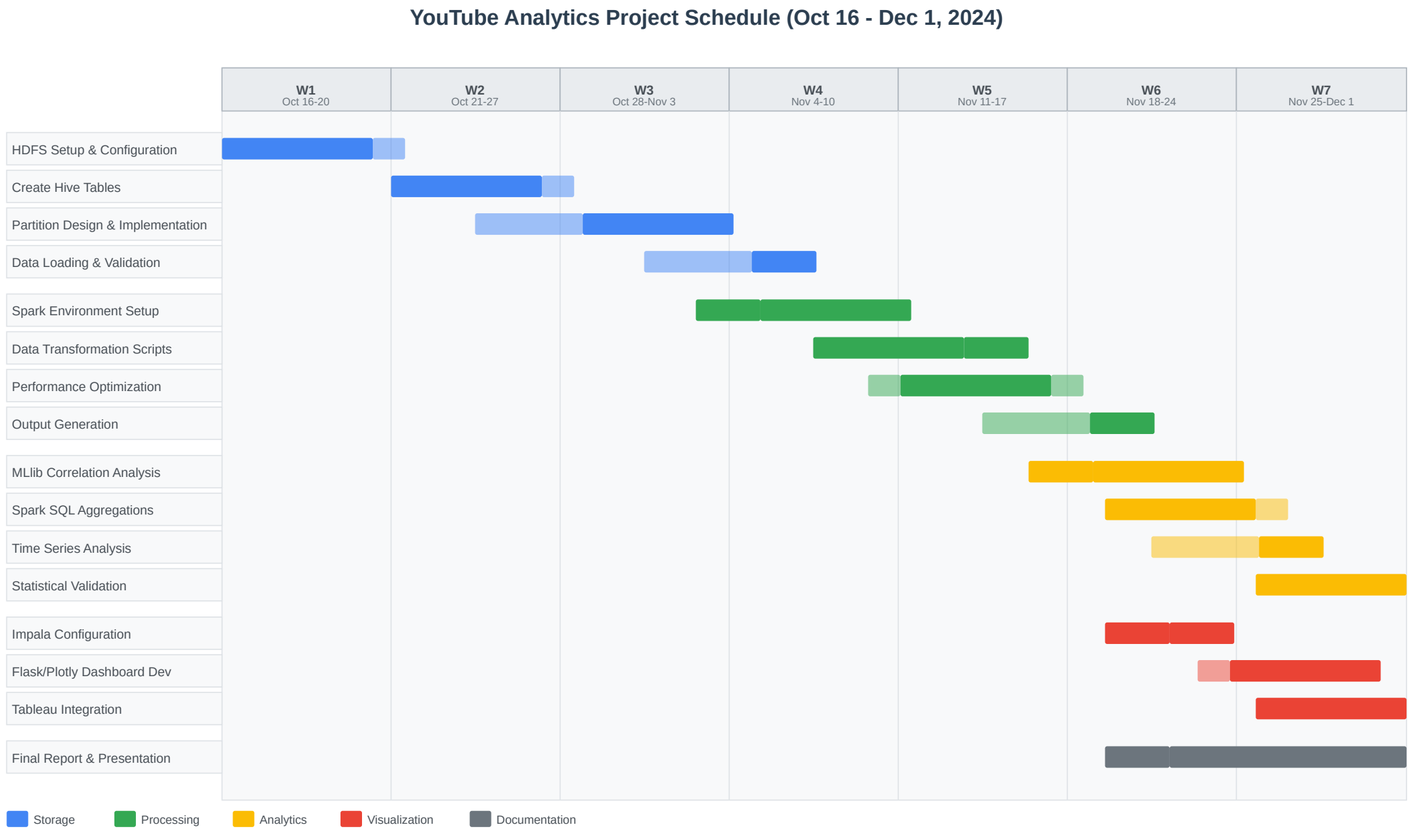
HARD TASKS

1. While still learning the structure of storing data for the first time, it was difficult to learn and perform the entire process of processing data directly.

2. After selecting the subject and sub-topic, data were collected, but it was difficult to verify the previously set sub-topic with the data, so the sub-topic had to be changed.

3. It was difficult to adapt to practical computing systems such as GCP.

PROJECT PLAN



- Work split：
  - First step：Collection and cleaning
  - Second step：Storage and Processing
  - Final step：Analysis and Visualization

Thank you